Generative AI & Retrieval-Augmented Generation

Sample Curriculum Document | Created for document-loader, text-splitter, and RAG testing

Document Information

Program name	Applied Generative AI Foundations
Duration	12 weeks
Study time	6–8 hours per week
Delivery format	Self-paced learning with practical labs
Prerequisites	Basic Python, command-line familiarity, and general programming concepts
Final project	A document question-answering assistant using RAG

Program Overview

This curriculum introduces the concepts and tools used to build applications powered by large language models (LLMs). Learners progress from Python and prompt design to document processing, embeddings, vector search, and Retrieval-Augmented Generation (RAG). Each module includes a practical task so that learners can test concepts in code.

Learning Outcomes

- Explain the difference between a language model response and a response grounded in retrieved documents.
- Create prompt templates and invoke a chat model using a Python library.
- Load text and PDF files and inspect their metadata.
- Split long documents into chunks while preserving useful context.
- Describe embeddings, vector stores, similarity search, and retrievers.
- Build a basic RAG pipeline and evaluate answers against source documents.

12-Week Course Plan

Week	Module	Topics	Practical task
1	Python Foundations	Functions, dictionaries, files, virtual environments, packages	Read a text file and extract its paragraphs
2	LLM Fundamentals	Tokens, context windows, inference, temperature, model limits	Compare responses using two prompt styles
3	Prompt Engineering	Instructions, context, examples, output constraints	Write prompts for summarization and extraction
4	LangChain Basics	Chat models, prompt templates, messages, invoke()	Build a prompt-template-to-model workflow
5	Runnable Composition	RunnableSequence, LCEL, parallel and passthrough patterns	Connect two runnable components

Week	Module	Topics	Practical task
6	Output Parsing	String output, JSON output, schema validation, error handling	Extract structured fields from a model response
7	Document Loaders	TextLoader, PyPDFLoader, CSVLoader, metadata	Load a PDF and inspect its pages
8	Text Splitters	Chunk size, chunk overlap, separators, recursive splitting	Split a long document and compare chunks
9	Embeddings	Vector representations, semantic similarity, embedding models	Generate embeddings for five sample passages
10	Vector Stores & Retrieval	Indexing, similarity search, top-k retrieval, metadata filters	Retrieve the three most relevant passages
11	RAG Pipeline	Question, retrieval, context assembly, grounded generation	Answer questions from a provided curriculum PDF
12	Evaluation & Final Project	Relevance, faithfulness, missing evidence, source attribution	Demo a document Q&A; assistant and document limitations

Module Details

Module A — Python Foundations

Schedule: Weeks 1

- Goal: prepare a clean Python environment and manipulate basic text data.
- Key concepts: variables, collections, functions, file I/O, exceptions, virtual environments, and package installation.
- Deliverable: a script that reads a text file, counts words, and reports the most frequent terms.

Module B — LLMs and Prompt Design

Schedule: Weeks 2–3

- Goal: understand model inputs and produce reliable instructions.
- Key concepts: tokens, context limits, system and user messages, few-shot examples, temperature, and constrained output.
- Deliverable: a small prompt library for summarization, question answering, and information extraction.

Module C — LangChain and Output Parsing

Schedule: Weeks 4–6

- Goal: compose reusable model workflows.
- Key concepts: chat model wrappers, prompt templates, runnable interfaces, LCEL, sequence composition, structured output, and validation.
- Deliverable: a chain that accepts a topic, generates a response, and returns validated fields.

Module D — Document Loaders

Schedule: Week 7

- Goal: convert source files into document objects suitable for downstream processing.
- Key concepts: page content, metadata, file encodings, PDF page boundaries, and loader-specific dependencies.
- Example loaders: TextLoader for plain text, PyPDFLoader for PDFs, and CSVLoader for CSV files.
- Deliverable: load a PDF and print each page's content and page metadata.

Module E — Text Splitters

Schedule: Week 8

- Goal: divide large documents into manageable, searchable chunks.
- Key concepts: chunk size, chunk overlap, separators, recursive splitting, and preserving metadata.
- A chunk that is too small may lose context. A chunk that is too large may reduce retrieval precision or exceed the model's context budget.
- Deliverable: compare chunk counts and sample text for chunk sizes of 300 and 700 characters.

Module F — Embeddings and Vector Search

Schedule: Weeks 9–10

- Goal: retrieve semantically relevant passages even when a question uses different wording from the source.

- Key concepts: embedding vectors, cosine similarity, vector stores, top-k results, metadata filters, and retriever interfaces.
- Deliverable: embed a collection of passages and retrieve the three closest matches for a question.

Module G — Retrieval-Augmented Generation

Schedule: Week 11

- Goal: generate answers using evidence retrieved from a knowledge collection.
- Typical flow: load documents, split them, create embeddings, index chunks, retrieve relevant chunks, construct a prompt with context, and invoke the LLM.
- RAG does not guarantee correctness. Retrieval can miss relevant evidence, and a model can still misinterpret the retrieved context.
- Deliverable: answer questions about this curriculum and identify when the document does not contain an answer.

Module H — Evaluation and Final Project

Schedule: Week 12

- Goal: evaluate whether answers are relevant, supported, and appropriately cautious.
- Evaluation checks: retrieval relevance, answer faithfulness, citation correctness, handling of unknown answers, and response consistency.
- Final project: create a document Q&A; assistant that accepts a PDF, indexes its text, answers questions, and shows the passages used as evidence.

Assessment Plan

Assessment	Weight	Description
Weekly exercises	25%	Short coding tasks and concept checks
Document processing lab	20%	Load documents, inspect metadata, and tune text splitting
Retrieval lab	20%	Create embeddings and inspect top-k retrieved chunks
Final RAG project	35%	Demonstrate document Q&A;, evidence use, and limitations

RAG Testing Dataset: Facts to Retrieve

The following facts are intentionally explicit so this PDF can be used to test document retrieval. Ask questions using both exact wording and paraphrases. These facts belong to this sample curriculum only; they are not claims about an external institution.

Question: What is the program name?
Expected answer: Applied Generative AI Foundations.

Question: How long is the program?
Expected answer: 12 weeks.

Question: How much study time is recommended each week?
Expected answer: 6–8 hours per week.

Question: What is the final project?
Expected answer: A document question-answering assistant using RAG.

Question: What should learners know before starting?
Expected answer: Basic Python, command-line familiarity, and general programming concepts.

Question: Which week covers document loaders?
Expected answer: Week 7.

Question: Which week covers text splitters?
Expected answer: Week 8.

Question: Which week introduces embeddings?
Expected answer: Week 9.

Question: When are vector stores and retrieval covered?
Expected answer: Week 10.

Question: When is the RAG pipeline covered?
Expected answer: Week 11.

Question: What is the weight of the final RAG project?
Expected answer: 35%.

Question: What is the weight of weekly exercises?
Expected answer: 25%.

Question: Which loader is listed for PDF files?
Expected answer: PyPDFLoader.

Question: Which loader is listed for CSV files?
Expected answer: CSVLoader.

Question: How many passages should the retrieval lab return?
Expected answer: The three closest matches.

Question: What are two risks of poor chunking?
Expected answer: Small chunks can lose context; large chunks can reduce retrieval precision or exceed the model context budget.

Question: Does RAG guarantee correctness?
Expected answer: No. Retrieval can miss relevant evidence, and the model can misinterpret retrieved context.

Question: What should the system do if the source lacks an answer?
Expected answer: It should state that the available document does not contain enough information rather than inventing an answer.

Sample Evaluation Questions

- Which module is scheduled for Week 8, and what is its primary goal?
- Compare the purposes of document loaders and text splitters.
- List the stages of the RAG pipeline in the order described in this document.
- What percentage of the assessment is assigned to the final project?
- Does this curriculum name a specific vector database vendor? If not, say that it does not specify one.
- What is the exact weekly study-time recommendation?
- Why should a RAG assistant avoid answering when retrieved evidence is insufficient?
- Which week covers vector stores and retrievers, and what practical task is assigned?

Important Note

This is a synthetic educational curriculum created for software testing. Its program details, schedule, and assessment weights are illustrative and should not be interpreted as the syllabus of a real college, university, or training provider.